Data Structures and Algorithms

Assignment 1 Sorting Algorithms

Prepared by

Youssef Ali Rashidy Ali
ID: 23010999

Faculty of Engineering
Alexandria University

March 6, 2026

Contents

1 Application Architecture **3**
1.1 Model . . . 3
1.2 View . . . 3
1.3 Controller . . . 4
1.4 Architectural Note . . . 4

2 Implementation: Class Hierarchy and Event-Driven Visualization **5**
2.1 Sorting Algorithm Hierarchy . . . 5
2.2 Event-Driven Visualization . . . 5
2.3 Observer Pattern for Event Recording and Operation Counting . . . 6
2.4 Algorithm Comparison and Benchmarking . . . 6
2.5 Summary . . . 6

3 View Architecture **7**

4 Controller Architecture **7**

5 Auxiliary Components and Design Patterns **8**
5.1 Array Loading and Generation . . . 8
5.2 Adapter Pattern . . . 9
5.3 Simple Factory Pattern . . . 9
5.4 Summary of Responsibilities . . . 9
5.5 Interfaces for Callbacks and Animation . . . 9

6 Assumptions and Design Choices **10**
6.1 Event-Driven Visualization . . . 10
6.2 Separation of Input Dimensions . . . 11
6.3 Immutable Event Representation . . . 12
6.4 Hybrid Benchmarking Approach . . . 12
6.5 Pre-recorded Visualization Sessions . . . 12

7 Data Aggregation **13**
7.1 Repeated Execution for Stable Measurements . . . 13
7.2 Operation Counting . . . 13
7.3 Aggregation Structure . . . 13
7.4 Preparing Data for Visualization and Analysis . . . 14
7.5 Sample of Raw Data . . . 14

8 Experimental Data Description **14**
8.1 Algorithms Evaluated 15
8.2 Input Distributions 15
8.3 Input Sizes 16
8.4 Sampling and Repeated Execution 16
8.5 Empirical Scaling of Sorting Algorithms 16
8.6 Log–Log Scaling Analysis 18
8.7 Algorithm Comparison by Input Size 19
8.8 Impact of Array Type on Algorithm Performance 20
8.9 Operation Counts on Random Arrays 21
8.10 Performance Stability Across Runs 22
8.11 Summary of Best-Performing Algorithms 23
8.12 Measured vs Theoretical Time Complexity 24

9 Sorting Visualization **25**
9.1 Input Configuration 25
9.2 Merge Sort Visualization 26
9.3 Quick Sort Visualization 27

10 Conclusion **28**
10.1 Design and Implementation 29
10.2 Experimental Findings 29
10.3 Lessons Learned 29

1 Application Architecture

The application follows the **Model–View–Controller (MVC)** design pattern. This architectural pattern separates the system into three main components in order to improve modularity, maintainability, and separation of concerns.

1.1 Model

The **Model** is responsible for the **core business logic** of the application. It is designed to be independent from the user interface and does not directly handle user interactions.

In this project, the Model is responsible for the following tasks:

- **Array loading**, either through random generation or by reading data from a file.
- **Implementation of sorting algorithms** used by the visualizer.
- **Generation of events** during algorithm execution that describe operations such as comparisons or swaps. These events are later used to drive the visualization.
- **Algorithm comparison and benchmarking**, allowing different sorting algorithms to be evaluated and compared using meaningful metrics such as execution time or number of operations.

The Model receives instructions from the **Controller**, performs the required computations, and produces the data necessary for visualization and analysis.

1.2 View

The **View** is responsible for **presenting data to the user and collecting user input**. It acts purely as a **presentation layer** and does not contain business logic.

Within the sorting visualizer, the View is responsible for:

- Displaying the array as graphical elements.
- Playing animations corresponding to sorting events.
- Providing the graphical interface through which users can select algorithms, generate arrays, load data, and control the visualization.

The View does not perform sorting or algorithmic processing; it only renders the results produced by the Model.

1.3 Controller

The **Controller** is responsible for **handling user interactions and coordinating communication between the Model and the View**.

When a user performs an action (such as generating an array, loading data from a file, or starting a sorting algorithm), the Controller:

1. Receives the user input from the View.
2. Invokes the appropriate operations in the Model.
3. Coordinates the flow of information required for visualization.

1.4 Architectural Note

It is important to note that the architecture used in this project is not a strict textbook implementation of MVC.

While the Model and the View remain largely separated, limited communication between them occurs indirectly through **callback functions** used by the visualization manager. These callbacks allow the View to react to events generated during algorithm execution without tightly coupling the visualization layer to the sorting logic.

This approach preserves the separation of responsibilities while enabling efficient real-time visualization of algorithm behavior.

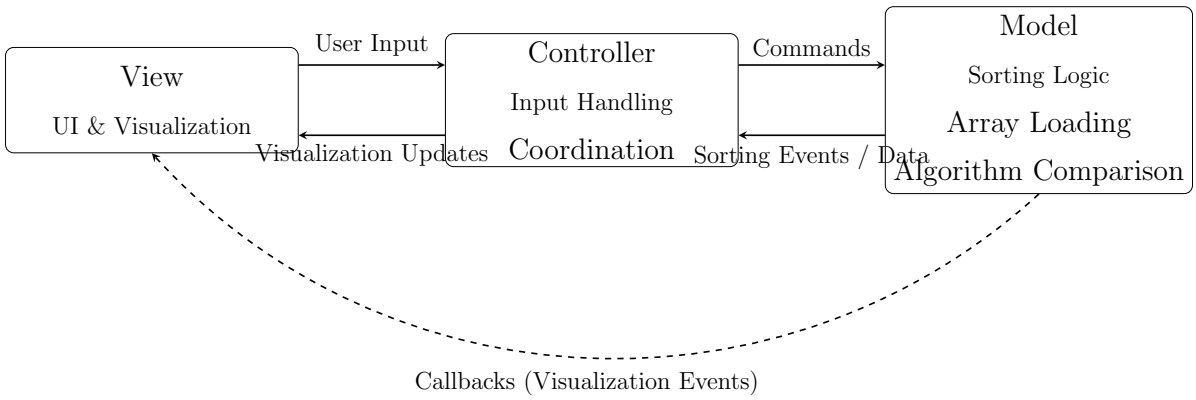


Figure 1: High-level architecture of the sorting visualizer following an MVC-inspired design. The dashed arrow represents limited indirect communication through callback functions used for visualization.

2 Implementation: Class Hierarchy and Event-Driven Visualization

The core of the application is designed around a **well-structured class hierarchy** and an **event-driven visualization approach**.

2.1 Sorting Algorithm Hierarchy

At the top of the hierarchy, the `SortingStrategy` interface defines the common contract for all sorting algorithms. This interface is implemented by concrete sorting algorithm classes, such as `BubbleSort`, `MergeSort`, and `QuickSort`.

For simulation purposes, a second-level abstract class, `AbstractSimulationAlgorithm`, inherits from `SortingStrategy` and provides shared functionality for event-driven execution. All simulation-oriented algorithm classes extend this abstract class to achieve the actual business logic while automatically generating events for visualization.

2.2 Event-Driven Visualization

Visualization in the application is **event-driven**. During algorithm execution, each meaningful operation triggers an event, which is then processed by the UI to animate the corresponding action. The system supports various types of events, including:

- **Comparison** and **Swap** events
- **Set** events for value assignment
- **Partition** and **Divide** events for recursive algorithms
- **Merge** and **MergeComparison** events for merge operations
- **Sorted** events indicating completion of a segment or the entire array

All events implement a **sealed marker interface `SortingEvent`** for uniform handling. Each event is implemented as a **record**, a special class that provides **immutable state**. This design ensures that once an event occurs, its data cannot be modified under any circumstances.

2.3 Observer Pattern for Event Recording and Operation Counting

To separate the concerns of algorithm execution from event management and operation counting, the **Observer design pattern** is employed. Each algorithm maintains a list of observers, which typically include:

- **EventRecorder**: listens to events and stores them for later playback in the visualization.
- **OperationCounter**: tracks the number of algorithm operations such as comparisons and swaps.

When an event occurs, the algorithm **notifies all observers**. Each observer responds independently to the notification, enabling the algorithm to remain focused solely on executing its logic without being responsible for event creation or counting operations. This design allows the system to easily extend with new observers in the future without modifying existing algorithm code.

2.4 Algorithm Comparison and Benchmarking

The `ComparisonManager` class is responsible for comparing multiple sorting algorithms. It executes each algorithm, collects operational data, and stores results in a custom data structure called `ComparisonState`. This structure records various parameters, including:

- Number of comparisons
- Number of swaps
- Execution time
- Other algorithm-specific metrics

The collected `ComparisonState` data is then sent to the Controller, which forwards it to the View for presentation in meaningful visual comparisons.

2.5 Summary

This architecture enables a **clean separation of concerns**:

- **Algorithms** focus purely on business logic.
- **Events and Observers** handle visualization and operation tracking.

- **ComparisonManager** provides benchmarking and uniform data collection.
- **Controller** orchestrates flow between Model and View, while the View visualizes data without knowledge of algorithmic logic.

By combining the **strategy pattern**, **abstract base classes**, **records for immutability**, and the **observer pattern**, the system is both modular and extensible, allowing additional algorithms, events, or observers to be added with minimal changes to existing code.

3 View Architecture

The view component of the application is responsible for presenting data to the user and collecting user input. The application consists of multiple views that are switched dynamically based on user interaction:

- **Main GUI:** This is the first view encountered when launching the application. It provides navigation to other views such as input, visualization, and comparison.
- **Input View:** Responsible for collecting user input, such as array size, type of data (random or from file), and algorithm selection. It forwards the input to the controller for processing.
- **Visualization View:** Displays the step-by-step execution of sorting algorithms. It reacts to events generated by the model to animate comparisons, swaps, and other algorithm-specific actions.
- **Comparison View:** Shows the results of comparing multiple sorting algorithms, typically in tabular or graphical form. It uses data provided by the comparison manager in the model.

It is important to note that the views are mostly passive and do not directly manipulate the business logic. The interaction between the view and the model is indirect and occurs through callback functions registered with the visualization manager, maintaining a limited coupling while enabling event-driven updates.

4 Controller Architecture

The controller component is responsible for handling all user inputs and coordinating communication between the view and the model. Its responsibilities can be summarized as follows:

- **Input Handling:** Receives user input encapsulated in a custom type `InputData` from the Input View.
- **Delegation:** Based on the type of input, the controller decides whether to delegate tasks to the `VisualizationManager` or the `ComparisonManager`.
- **Visualization Coordination:** When visualization is requested, the controller divides the task into sessions. This allows the application to visualize multiple algorithms and/or multiple arrays sequentially. A callback function is sent to the `VisualizationManager` to notify the controller once a session has ended.
- **Comparison Coordination:** When comparison is requested, the controller forwards the input to the `ComparisonManager`, which processes the data and returns a `LinkedHashMap` containing the results. The controller then forwards these results to the `ComparisonView`.

It is important to note that the controller acts as the main coordination hub of the application. While the view and the model may communicate indirectly, the controller ensures that all user interactions are correctly routed and that the flow of data remains consistent.

5 Auxiliary Components and Design Patterns

The application relies on several auxiliary components and design patterns to support array management, algorithm creation, and data adaptation for comparison. These components ensure modularity and facilitate maintainability.

5.1 Array Loading and Generation

Array loading and generation are part of the model layer, responsible for providing the data required by sorting algorithms and comparison operations:

- **File Reader:** Loads arrays from one or multiple files specified by the user. It parses the file(s) and returns arrays for further processing by the model.
- **Array Generator:** Generates arrays with different characteristics:
  - *Random:* Fully randomized values.
  - *Sorted:* Pre-sorted in ascending order.
  - *Inversely Sorted:* Pre-sorted in descending order.

- *Nearly Sorted:* Mostly sorted with a few random disruptions.

The generator also supports batch generation of multiple arrays, which is useful for visualization and comparison sessions.

5.2 Adapter Pattern

The application uses the **Adapter Pattern** to bridge the array loading components with the comparison manager. The arrays returned by the file reader or generator are adapted into `ComparisonArray` objects, which include metadata such as generation mode and source file. This ensures uniform treatment by the comparison manager without changing its internal implementation.

5.3 Simple Factory Pattern

A **Simple Factory Pattern** is used to simplify the creation of different sorting algorithms. The factory provides a single interface to instantiate any algorithm, whether it is a pure sorting algorithm or a simulation algorithm. This approach improves modularity and allows easy extension with new algorithms in the future.

5.4 Summary of Responsibilities

- **File Reader:** Load arrays from file paths.
- **Array Generator:** Generate arrays of various types and sizes, including batches.
- **Adapter:** Convert loaded or generated arrays into a uniform format for the comparison manager.
- **Algorithm Factory:** Provide a convenient interface to create any sorting algorithm dynamically.

5.5 Interfaces for Callbacks and Animation

The application defines several interfaces to facilitate communication, animation control, and semantic clarity:

- **SessionCallback:** A functional interface used to signal the completion of a visualization session. This allows the controller or other components to register a lambda function to be executed once a session ends. Although a simple `Runnable` could suffice, this interface provides semantic meaning within the context of the application.

- **AnimationCallback:** Another functional interface, mainly used to trigger updates in the view when an animation step occurs. It allows lambda expressions to be passed for custom handling of animation events, providing flexibility while maintaining readability.

- **Animatable:** An interface that promises that any implementing class supports animation controls, specifically `play`, `pause`, and `stop` methods. This is used to standardize animation control across different visual components and ensures consistent behavior in visualization views.

These interfaces provide modularity, semantic clarity, and flexibility in handling event-driven visualization and user interactions, complementing the MVC architecture and the observer-driven event system of the model.

6 Assumptions and Design Choices

This section highlights several key design decisions that influenced the implementation of the sorting visualizer. These choices were made to improve reliability, flexibility, and maintainability of the system.

6.1 Event-Driven Visualization

The visualization system follows an **event-driven approach**. During algorithm execution, each significant operation such as a comparison, swap, or assignment generates a corresponding event.

These events are recorded and later consumed by the visualization layer to produce animations.

This design was chosen for several reasons:

- It avoids synchronization issues that may arise when running sorting algorithms concurrently with the JavaFX UI thread.

- It allows precise control over animation speed.

- It enables predictable pause, resume, and step-by-step execution.

- It simplifies the implementation of reset functionality by replaying recorded events.

Using threads to directly animate algorithms would make synchronization and control significantly more complex.

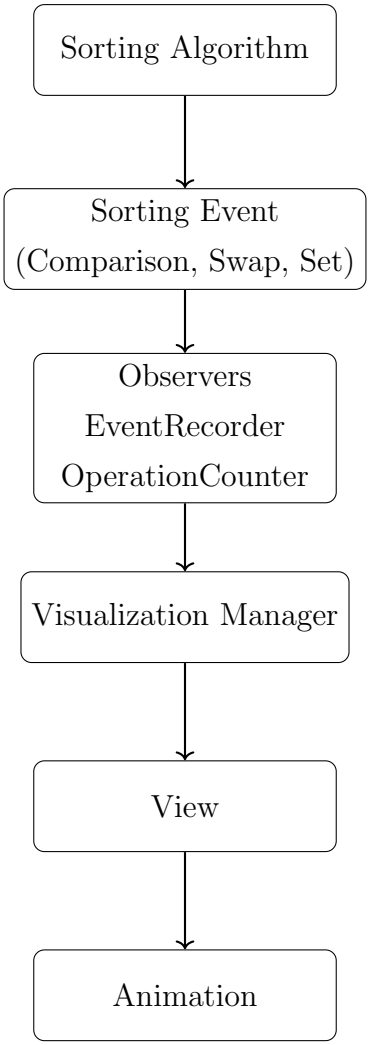


Figure 2: Event-driven visualization pipeline used by the sorting visualizer. Sorting algorithms generate events which are processed by observers and later replayed by the visualization manager to produce animations in the view.

6.2 Separation of Input Dimensions

User input is modeled using two independent dimensions:

- **Array Size**
- **Generation Mode**

For each selected array size, the system generates one array for every selected generation mode. Examples of generation modes include random, sorted, inversely sorted, and nearly sorted arrays.

This approach allows systematic experimentation by varying one dimension independently from the other, which simplifies both visualization sessions and algorithm comparisons.

6.3 Immutable Event Representation

All sorting events are implemented as **Java records**. Records provide immutable data structures whose state cannot be modified after creation.

Immutability was chosen for the following reasons:

- Events represent operations that already occurred during algorithm execution.
- Preventing modification ensures the integrity of recorded execution traces.
- Immutable objects simplify reasoning about program behavior and reduce the risk of unintended side effects.

6.4 Hybrid Benchmarking Approach

To obtain reliable performance measurements, the comparison mode uses a hybrid strategy:

- **Simulation algorithms** are used to count operations such as comparisons and swaps.
- The same arrays are then sorted using **pure sorting implementations** to measure execution time.

In order to reduce noise in execution time measurements, each array is sorted **ten times**, and the measured times are aggregated to obtain a more stable estimate of algorithm performance.

This approach separates visualization-related logic from timing measurements, ensuring that the collected performance metrics reflect the actual behavior of the algorithms.

6.5 Pre-recorded Visualization Sessions

Before visualization begins, sorting algorithms are executed once in order to **record the sequence of events** generated during execution.

The visualization layer then replays these events to animate the algorithm.

Pre-recording events offers several advantages:

- Animations remain smooth because no algorithm computation occurs during playback.
- The animation speed can be adjusted independently of algorithm execution.
- Features such as pause, resume, and reset become easier to implement.

This design ensures that visualization behavior remains predictable and responsive to user interaction.

7 Data Aggregation

In order to produce meaningful comparisons between sorting algorithms, raw execution data must be aggregated into structured results. The system performs multiple runs of each algorithm and collects various metrics, which are then summarized and presented to the user.

7.1 Repeated Execution for Stable Measurements

Execution time measurements are often affected by factors such as system load, JVM optimizations, and background processes. To reduce this noise, each sorting algorithm is executed multiple times on the same input array.

In this project, each array is sorted **ten times**. The collected execution times are then aggregated to produce a more stable estimate of the algorithm's performance.

This repeated execution improves the reliability of the comparison results and prevents conclusions from being based on a single noisy measurement.

7.2 Operation Counting

In addition to execution time, the system also measures algorithmic operations. These include:

- Number of comparisons
- Number of swaps
- Other algorithm-specific operations

These metrics are collected using the `OperationCounter` observer, which listens to sorting events generated during simulation execution. Because the counting process relies on the same events used for visualization, the measured operations accurately reflect the internal behavior of the algorithms.

7.3 Aggregation Structure

The collected metrics are stored in instances of the `ComparisonState` class. Each instance represents the aggregated results for a specific algorithm on a specific input configuration.

A `LinkedHashMap` is used to store the results produced by the `ComparisonManager`. The use of this data structure preserves the insertion order, ensuring that algorithms appear in the output in the same order selected by the user.

7.4 Preparing Data for Visualization and Analysis

After aggregation, the results are forwarded to the controller and then to the comparison view. The view presents the data in tabular or graphical form, allowing users to easily compare algorithms across different input sizes and generation modes.

This aggregation process transforms raw execution data into structured information that can be analyzed and interpreted effectively.

7.5 Sample of Raw Data

The following table presents a representative sample of the collected dataset, illustrating the structure of the recorded observations.

Algorithm	Size	Mode	Runs	Avg Time (ns)	Min Time (ns)	Max Time (ns)	Comparisons	Interchanges
Bubble Sort	10000	INVERSELY_SORTED	10	88,344,930	83,231,200	99,570,200	49,995,000	49,994,524
Heap Sort	1000	INVERSELY_SORTED	10	94,210	69,400	205,900	24,782	8,316
Insertion Sort	2500	SORTED	10	4,270	4,200	4,600	2,499	0
Insertion Sort	10	RANDOM	10	650	100	3,400	16	8
Insertion Sort	10	NEARLY_SORTED	10	810	300	2,800	32	24
Merge Sort	100	NEARLY_SORTED	10	35,320	28,600	77,400	545	672
Merge Sort	1000	NEARLY_SORTED	10	175,580	145,500	190,100	8,708	9,976
Merge Sort	2500	RANDOM	10	756,090	441,600	1,529,300	25,114	28,404
Merge Sort	250	INVERSELY_SORTED	10	32,290	28,900	37,900	983	1,994
Merge Sort	10	RANDOM	10	2,060	1,700	3,100	25	34
Merge Sort	100	SORTED	10	172,550	63,400	258,300	356	672
Merge Sort	5000	SORTED	10	382,420	295,600	471,400	32,004	61,808
Quick Sort	500	INVERSELY_SORTED	10	33,990	29,200	45,500	5,568	3,572
Quick Sort	10000	INVERSELY_SORTED	10	551,780	501,200	749,400	152,021	90,499
Quick Sort	5000	NEARLY_SORTED	10	548,490	380,600	773,500	72,055	41,842
Quick Sort	500	NEARLY_SORTED	10	41,230	38,300	45,400	5,034	3,364
Quick Sort	5000	INVERSELY_SORTED	10	337,900	245,000	480,400	75,214	43,753
Selection Sort	10	INVERSELY_SORTED	10	970	300	2,900	45	10
Selection Sort	2500	NEARLY_SORTED	10	2,208,090	1,976,400	2,742,000	3,123,750	2,500
Selection Sort	1000	NEARLY_SORTED	10	643,670	521,700	895,800	499,500	1,000

Table 1: Sample of 20 randomly selected observations from the collected dataset (2160 total). Times are recorded in nanoseconds.

8 Experimental Data Description

The empirical analysis of the sorting algorithms was conducted using data collected from multiple benchmark executions. The dataset contains **2160 observations**, where each observation corresponds to the execution of a sorting algorithm under a specific input configuration.

Each configuration is defined by three parameters:

- **Sorting Algorithm**
- **Input Generation Mode**
- **Input Size**

For every combination of these parameters, the experiment uses multiple independent input samples and repeated executions in order to produce reliable measurements.

8.1 Algorithms Evaluated

The following sorting algorithms were included in the experiments:

- Bubble Sort
- Insertion Sort
- Selection Sort
- Merge Sort
- Heap Sort
- Quick Sort

These algorithms were selected to represent both *quadratic-time algorithms* and more efficient $O(n \log n)$ algorithms.

8.2 Input Distributions

Arrays were generated using four different distributions in order to evaluate algorithm behavior under different input characteristics:

- **Random**: values are uniformly randomized.
- **Sorted**: the array is already in ascending order.
- **Inversely Sorted**: the array is in descending order.
- **Nearly Sorted**: the array is mostly sorted with a small number of random perturbations.

Using multiple input distributions allows the experiments to capture both typical and edge-case behavior for different algorithms.

8.3 Input Sizes

The experiments were conducted using the following array sizes:

$$10,\ 50,\ 100,\ 250,\ 500,\ 1000,\ 2500,\ 5000,\ 10000$$

This range allows the study of algorithm performance from very small inputs to moderately large inputs, making it possible to observe how running time grows with respect to n.

8.4 Sampling and Repeated Execution

To avoid bias caused by a single randomly generated array, each combination of input size and generation mode produces **ten independent arrays**. These arrays represent different instances drawn from the same distribution.

Each array is then processed by all sorting algorithms included in the experiment. For performance measurements, every algorithm is executed **ten times on the same array**. The execution times are then averaged to obtain a more stable estimate of runtime.

This process reduces measurement noise caused by factors such as system load or runtime optimizations.

The total number of observations recorded in the dataset is therefore:

$$6 \times 4 \times 9 \times 10 = 2160$$

where:

- 6 is the number of algorithms,
- 4 is the number of input distributions,
- 9 is the number of input sizes,
- 10 represents the number of independent arrays generated for each configuration.

8.5 Empirical Scaling of Sorting Algorithms

To study how the execution time of each algorithm grows with the input size, the collected measurements were first aggregated. For each combination of algorithm, input size, and input distribution, the following statistics were computed:

- Median execution time
- Minimum execution time

- Maximum execution time
- Median number of comparisons
- Median number of interchanges

The median was used instead of the mean in order to reduce the influence of occasional outliers in runtime measurements.

After aggregation, the data was grouped by algorithm and input size, and the median execution time across different input modes was computed. This allows the visualization of the overall scaling behavior of each sorting algorithm as the input size increases.

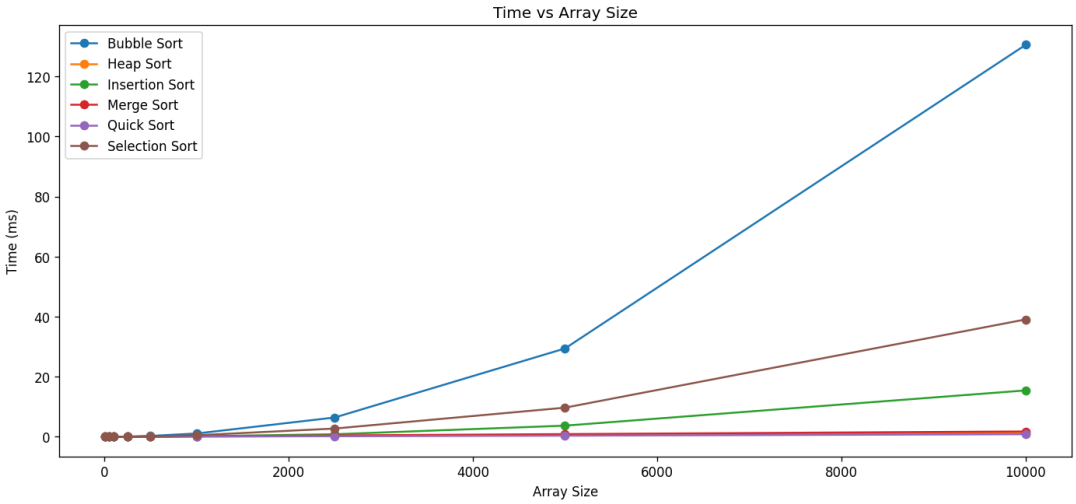


Figure 3: Execution time as a function of array size for different sorting algorithms.

Figure 3 shows the empirical growth of execution time for the evaluated algorithms as the array size increases.

The results clearly illustrate the differences between quadratic-time algorithms and more efficient $O(n \log n)$ algorithms. **Bubble Sort** exhibits the worst scaling behavior, with execution time increasing rapidly as the input size grows. **Selection Sort** also demonstrates quadratic growth, although it performs slightly better than Bubble Sort in practice.

Insertion Sort performs better than both Bubble Sort and Selection Sort , which is expected due to its relatively low constant factors and adaptive behavior. However, as the input size increases, its quadratic nature becomes evident.

In contrast, the algorithms **Merge Sort**, **Heap Sort**, and **Quick Sort** scale significantly better. Their growth pattern follows the expected $O(n \log n)$ complexity, resulting in much slower increases in execution time compared to quadratic algorithms.

Overall, the figure empirically confirms the theoretical complexity differences between the studied sorting algorithms.

8.6 Log–Log Scaling Analysis

To better visualize the asymptotic growth of the algorithms, the same data was plotted using a **log–log scale**. Both the array size and execution time axes were transformed using logarithmic scaling.

This transformation is particularly useful for algorithm analysis because polynomial relationships appear as approximately straight lines. In such plots, the **slope of the line corresponds to the exponent of the time complexity**. For example:

- Quadratic algorithms $O(n^2)$ should exhibit a slope close to 2.
- $O(n \log n)$ algorithms should exhibit a slope slightly above 1.

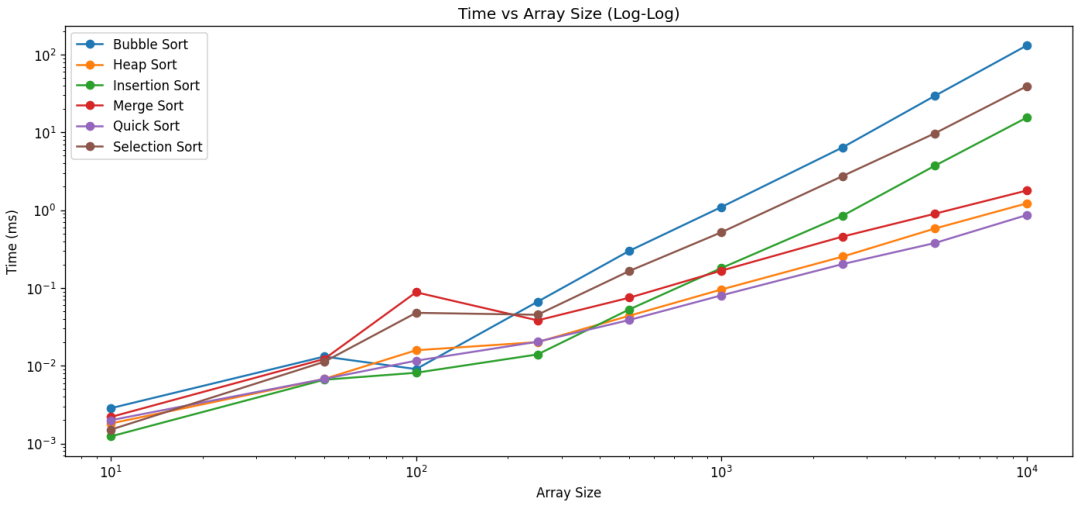


Figure 4: Log–log plot of execution time versus array size for different sorting algorithms.

The log–log representation makes the separation between algorithmic complexity classes significantly clearer. The quadratic algorithms — **Bubble Sort**, **Selection Sort**, and **Insertion Sort** — exhibit noticeably steeper slopes compared to the more efficient algorithms.

In contrast, **Merge Sort**, **Heap Sort**, and **Quick Sort** show much gentler slopes, consistent with their expected $O(n \log n)$ complexity. The near-linear appearance of these curves in the log–log plot confirms that their growth rate increases far more slowly than the quadratic algorithms.

Overall, the log–log visualization provides empirical evidence supporting the theoretical complexity analysis of the sorting algorithms.

8.7 Algorithm Comparison by Input Size

In order to compare algorithm performance more intuitively for individual array sizes, a series of bar charts were generated. Each chart shows the median execution time of all sorting algorithms for a specific input size.

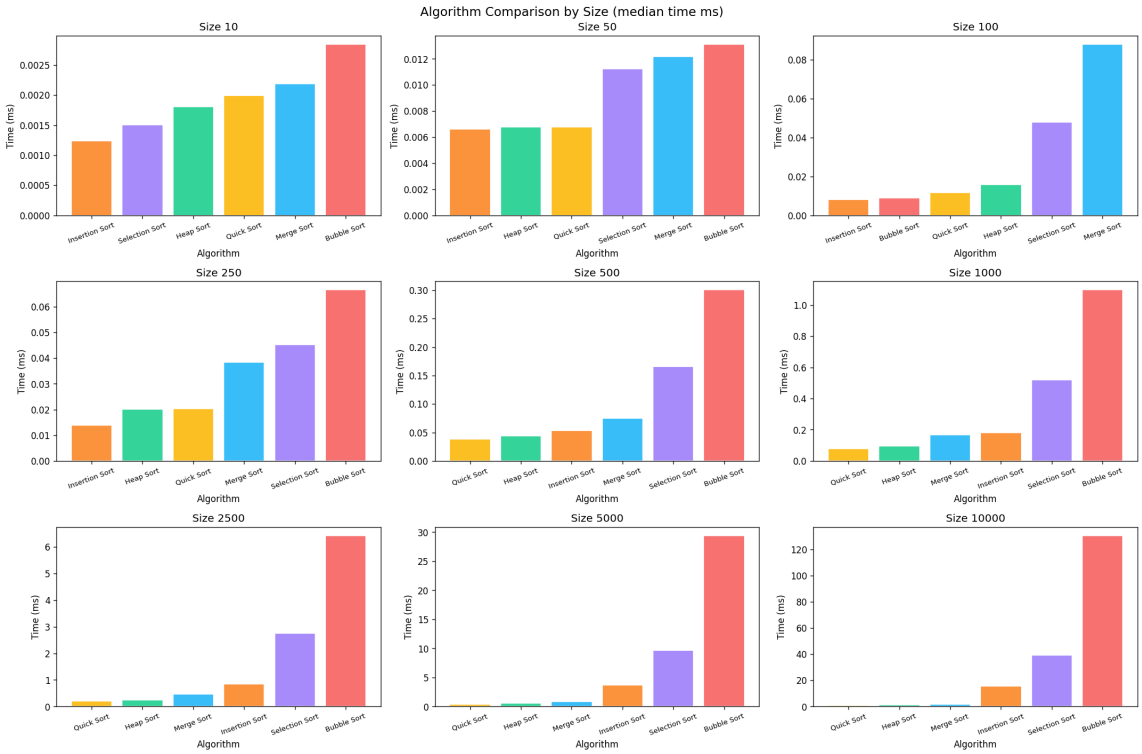


Figure 5: Median execution time (ms) of different sorting algorithms for each array size.

The bar charts allow direct comparison between algorithms for each input size. Key observations include:

- **Bubble Sort** generally performs worse than other algorithms, except for one case where it is highly probable that the random input array was unusually favorable due to the use of an optimized version.
- For small input sizes, **Insertion Sort** and **Selection Sort** perform better than **Merge Sort**, which is justifiable because of constant factors.
- For large array sizes, the distinction is clear: $O(n \log n)$ algorithms dominate, where their run time is roughly negligible compared to the $O(n^2)$ algorithms.

These bar charts complement the line and log–log plots by providing an immediate, size-by-size ranking of algorithm performance, reinforcing the trends observed in earlier analyses.

8.8 Impact of Array Type on Algorithm Performance

To investigate how different input distributions affect algorithm performance, a series of grouped bar charts was generated. Each chart shows the median execution time of all sorting algorithms for a specific array size, with separate bars representing different array types: *Random*, *Sorted*, *Inversely Sorted*, and *Nearly Sorted*.

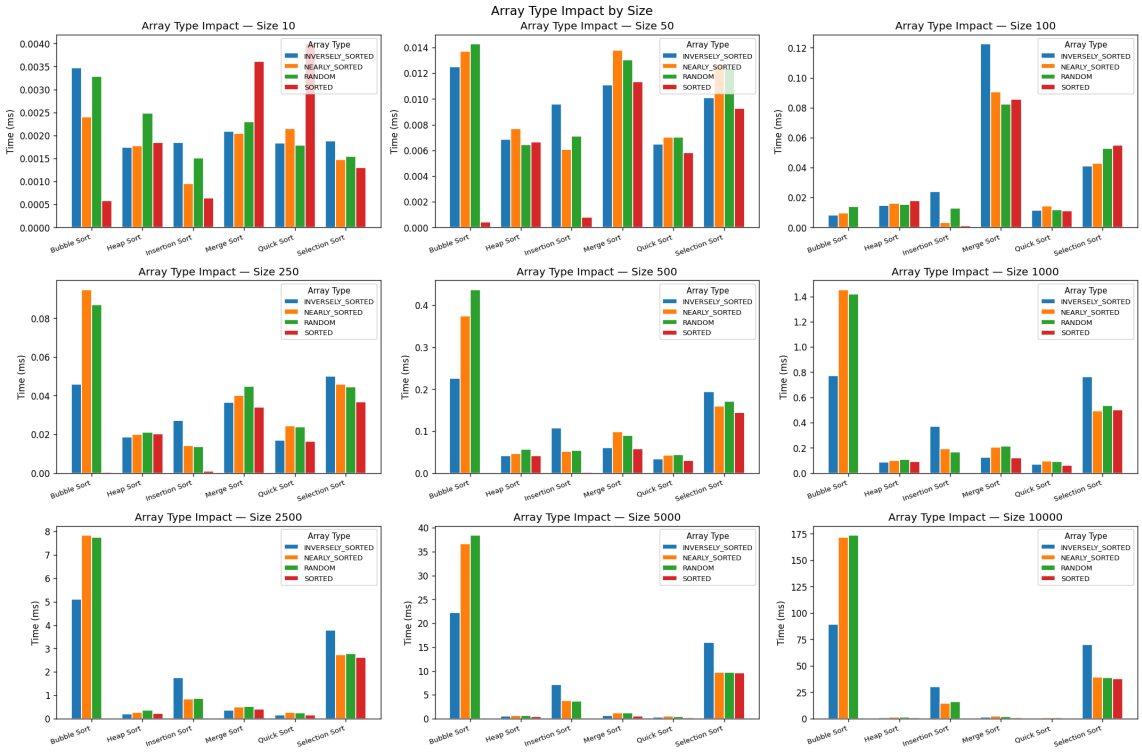


Figure 6: Impact of array type on execution time across different sorting algorithms and array sizes.

The results reveal several interesting patterns:

- **Bubble Sort** exhibits an unusual trend where its performance on the worst-case input is sometimes better than on random or nearly sorted arrays. This is likely due to optimizations in the implementation that can detect early termination in certain arrangements.

- **Insertion Sort** and **Selection Sort** show expected behavior: their performance is significantly worse on inversely sorted arrays, while they perform better on nearly sorted or sorted inputs, consistent with their adaptive and non-adaptive characteristics, respectively.

- $O(n \log n)$ algorithms (**Merge Sort**, **Heap Sort**, and **Quick Sort**) are largely unaffected by array type, demonstrating high consistency across input distributions. This is justified because their procedures are largely input-agnostic:

- Merge Sort always divides the array into subproblems and merges them systematically.
- Heap Sort always constructs a max-heap before performing heap operations.
- Quick Sort selects a pivot (randomized in this implementation) and performs the same comparison procedure regardless of initial order.

Consequently, the number of operations for $O(n \log n)$ algorithms remains relatively stable across different array types.

These observations highlight the influence of input data characteristics on algorithm performance and confirm theoretical expectations for both quadratic and $O(n \log n)$ algorithms.

8.9 Operation Counts on Random Arrays

To complement the runtime analysis, the number of algorithmic operations — **comparisons** and **interchanges** — was measured for all sorting algorithms on random input arrays. Grouped bar charts were generated for each array size, displaying the median counts of comparisons and interchanges.

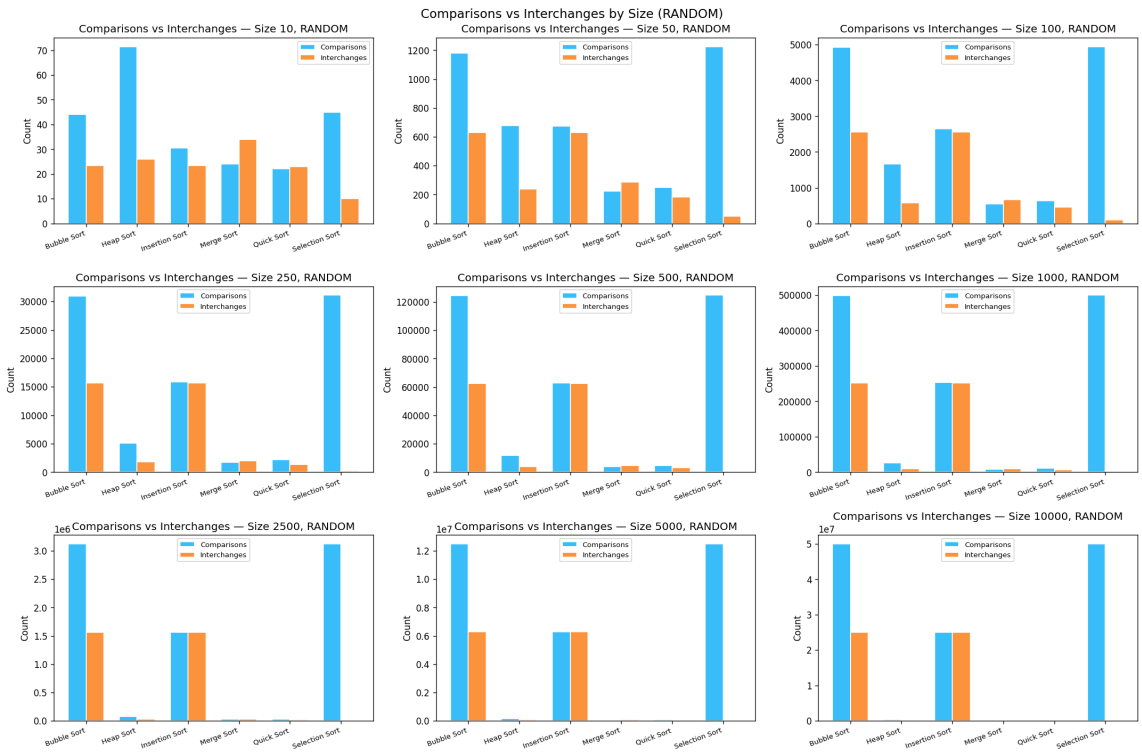


Figure 7: Comparisons and interchanges for different sorting algorithms on random input arrays across various sizes.

The results indicate several patterns:

- **Bubble Sort, Heap Sort, and Selection Sort** are dominated by comparisons, meaning most of their execution time is spent performing comparisons rather than swaps.
- **Insertion Sort** and **Quick Sort** exhibit nearly equal numbers of comparisons and interchanges, reflecting their balanced behavior in both types of operations. Heap Sort shows a similar trend for interchanges when building and maintaining the heap.
- **Merge Sort** is dominated by interchanges, as its divide-and-conquer approach requires frequent merging of subarrays, which involves moving elements rather than comparing them extensively.

These observations complement the timing analysis by providing a deeper understanding of how each algorithm's internal operations contribute to overall performance. They also highlight that different algorithms have varying operational bottlenecks: some are comparison-heavy, while others are interchange-heavy.

8.10 Performance Stability Across Runs

To assess the consistency of algorithm performance, the variance of execution time across repeated runs was calculated for each algorithm and array size. Specifically, variance was computed as the difference between the maximum and minimum measured execution times for each array, and then the median across input types was taken.

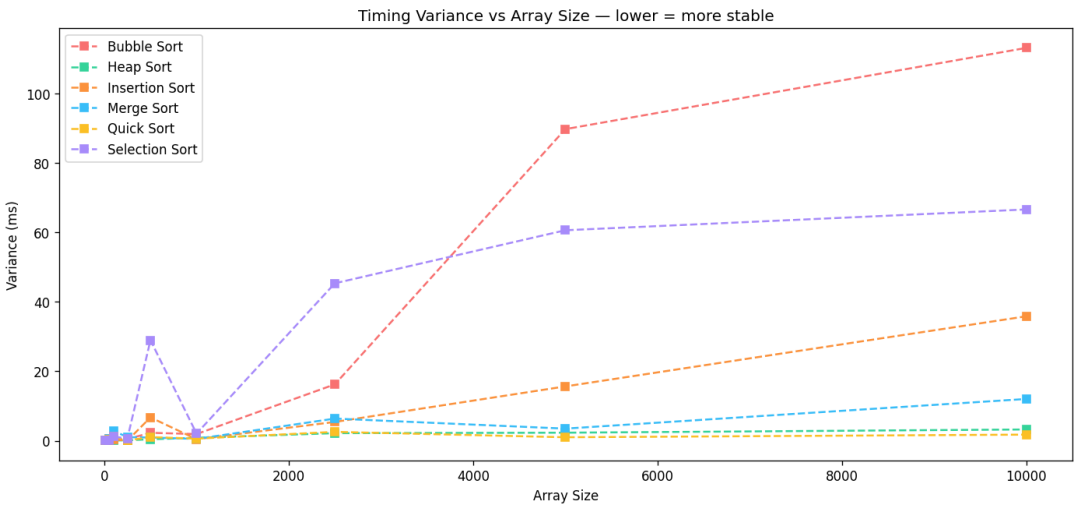


Figure 8: Execution time variance vs. array size for different sorting algorithms. Lower variance indicates more stable performance.

Key observations include:

- **Bubble Sort** exhibits the highest variance, indicating its execution time is highly sensitive to the input instance. **Selection Sort** also shows relatively high variance, though less extreme.
- **Insertion Sort** performs better than both Bubble and Selection Sort in terms of stability, especially for smaller input sizes.
- $O(n \log n)$ algorithms (**Merge Sort**, **Heap Sort**, and **Quick Sort**) demonstrate a high degree of stability, with relatively low variance across repeated runs, reflecting their predictable operational structure and input-agnostic procedures.

This analysis complements both runtime and operation count analyses by highlighting not only how fast algorithms are on average, but also how consistently they perform under repeated executions.

8.11 Summary of Best-Performing Algorithms

To provide a concise overview of the experimental results, the best-performing algorithm for each combination of array size and input type is summarized in the table below. The selection is based on median execution time across multiple runs.

Size	INVERSELY_SORTED	NEARLY_SORTED	RANDOM	SORTED
10	Heap Sort	Insertion Sort	Insertion Sort	Bubble Sort
50	Quick Sort	Insertion Sort	Heap Sort	Bubble Sort
100	Bubble Sort	Insertion Sort	Quick Sort	Bubble Sort
250	Quick Sort	Insertion Sort	Insertion Sort	Bubble Sort
500	Quick Sort	Quick Sort	Quick Sort	Bubble Sort
1000	Quick Sort	Quick Sort	Quick Sort	Bubble Sort
2500	Quick Sort	Heap Sort	Quick Sort	Bubble Sort
5000	Quick Sort	Quick Sort	Quick Sort	Bubble Sort
10000	Quick Sort	Quick Sort	Quick Sort	Bubble Sort

Table 2: Best-performing algorithms by input size and array type based on median execution time.

This table highlights several trends:

- **Quick Sort** dominates most medium and large array sizes across all input types, reflecting its strong asymptotic performance.

- **Insertion Sort** performs best for small or nearly sorted arrays due to low constant factors and adaptive behavior.
- **Bubble Sort** only occasionally appears as the best performer, primarily for very small sorted arrays or lucky cases with optimized early termination.
- The choice of algorithm becomes increasingly critical as array size grows, with $O(n \log n)$ algorithms clearly outperforming quadratic algorithms for larger inputs.

8.12 Measured vs Theoretical Time Complexity

To validate the experimental results against theoretical expectations, each algorithm's measured execution time was plotted alongside a scaled version of its asymptotic time complexity. The theoretical curves were scaled to match the maximum measured time for visual comparison.

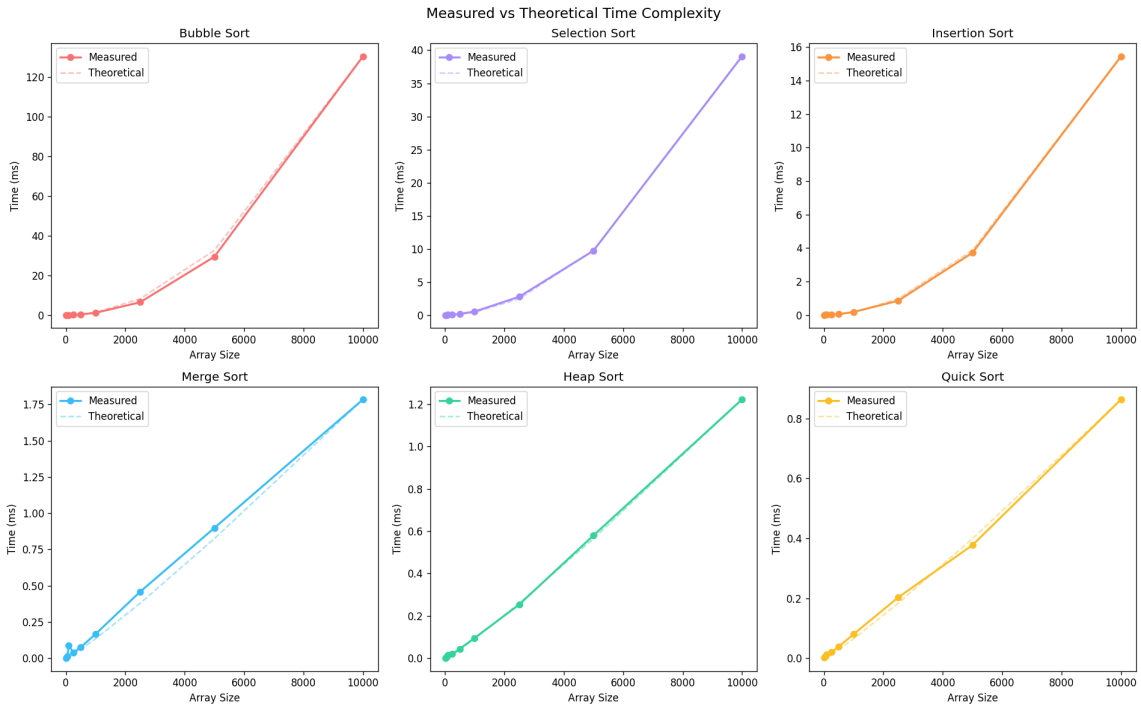


Figure 9: Measured execution time vs. theoretical time complexity for each sorting algorithm. Solid lines represent experimental median execution times, while dashed lines show the corresponding scaled theoretical curves.

Key observations include:

- Measured run times closely follow the expected asymptotic trends for each algorithm, confirming the correctness of the implementations.
- Quadratic algorithms (**Bubble Sort**, **Insertion Sort**, **Selection Sort**) exhibit steep growth as array size increases, consistent with their $O(n^2)$ complexity.

- $O(n \log n)$ algorithms (**Merge Sort**, **Heap Sort**, **Quick Sort**) display significantly slower growth in execution time for large inputs, validating their superior asymptotic performance.
- Minor deviations between measured and theoretical curves occur due to constant factors, memory hierarchy effects, and random variations in input arrays, but the overall trends remain highly consistent.

This comparison provides a final confirmation that the empirical results align well with theoretical expectations and supports the conclusions drawn from earlier runtime, operation count, array type, and stability analyses.

9 Sorting Visualization

This section presents sample outputs from the sorting visualizer, demonstrating the application's graphical interface and its ability to animate sorting algorithms in real time. The visualization was tested using two algorithms: **Merge Sort** and **Quick Sort**.

9.1 Input Configuration

Before launching a visualization session, the user configures the input parameters through the input view. This includes selecting the sorting algorithm, specifying the array size, and choosing the array generation mode.

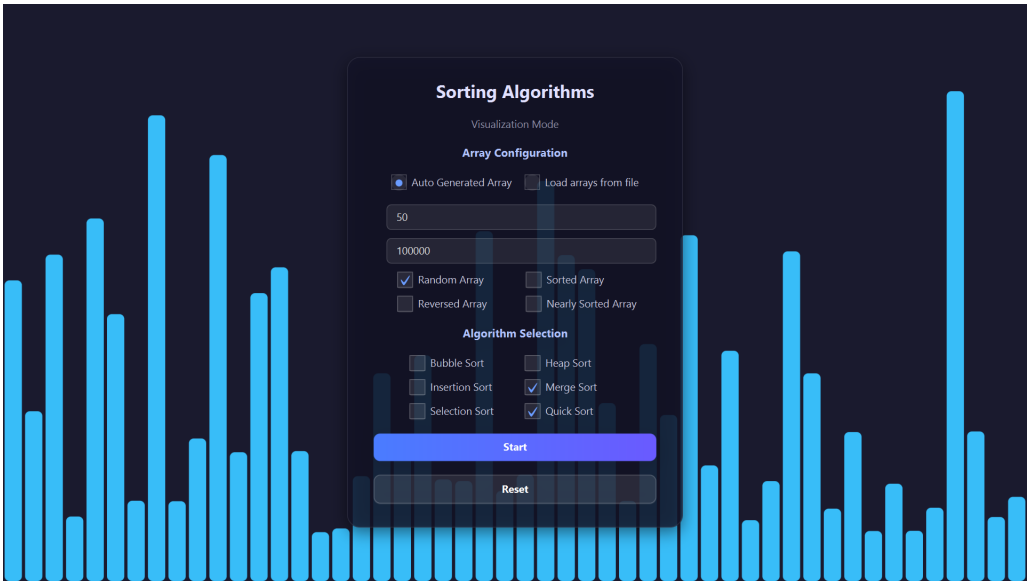
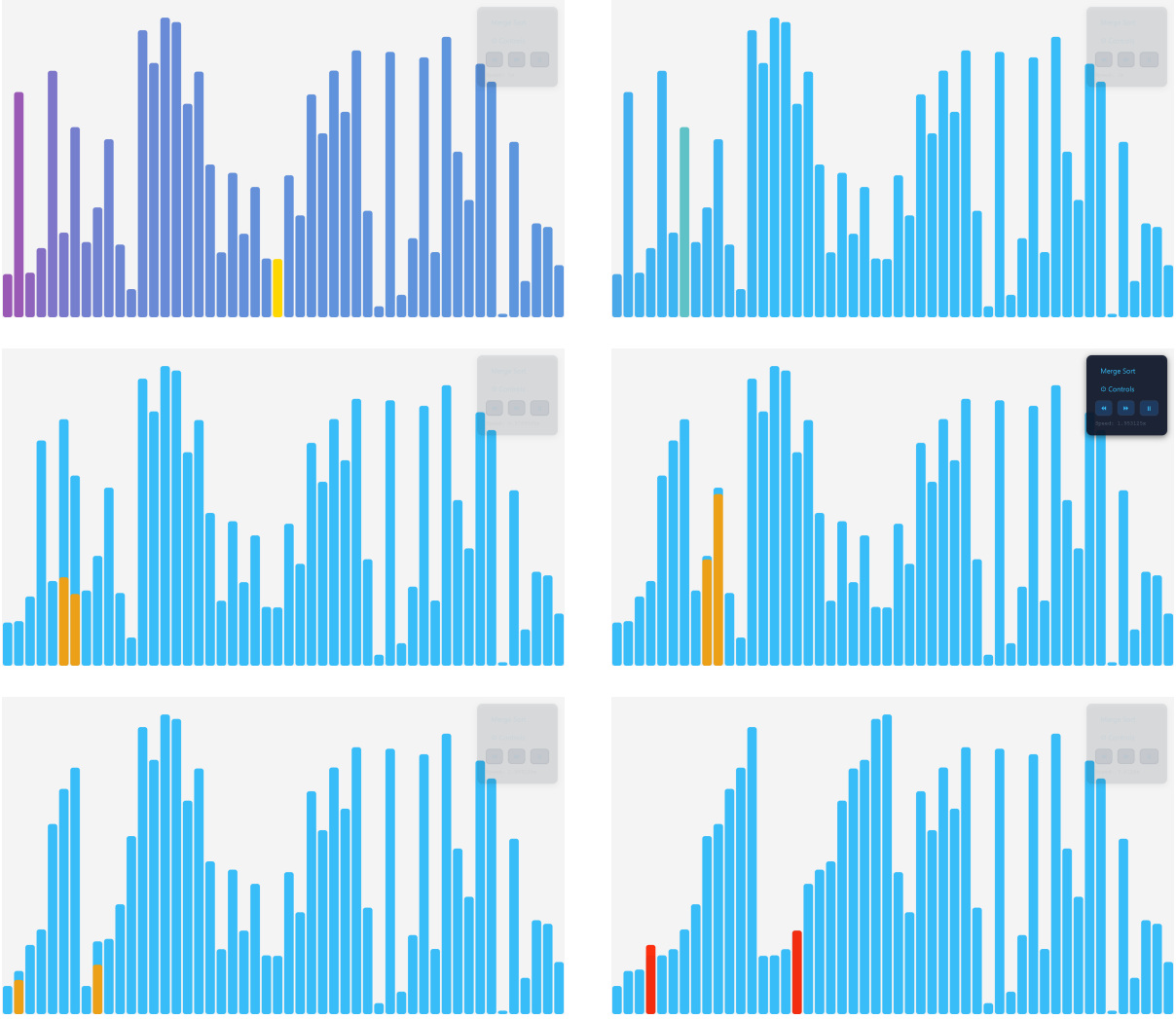
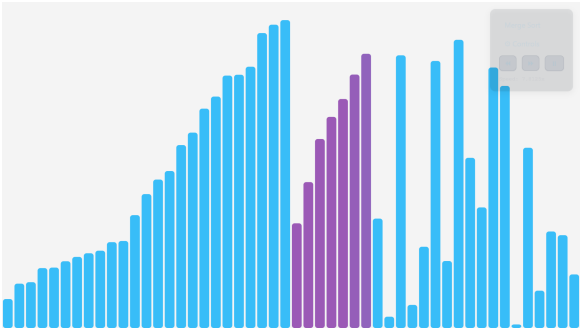


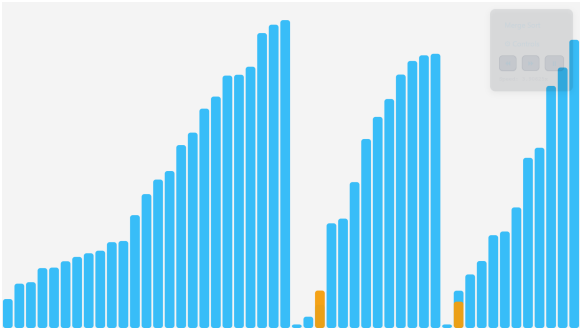
Figure 10: Input configuration screen for the sorting visualizer.

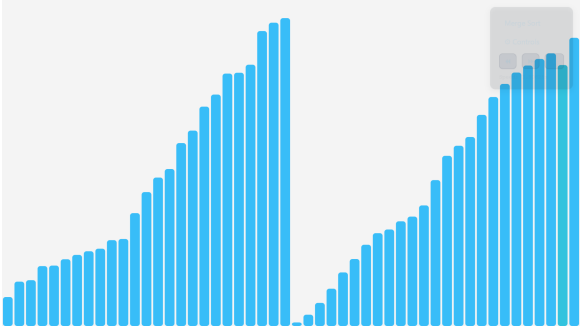
9.2 Merge Sort Visualization

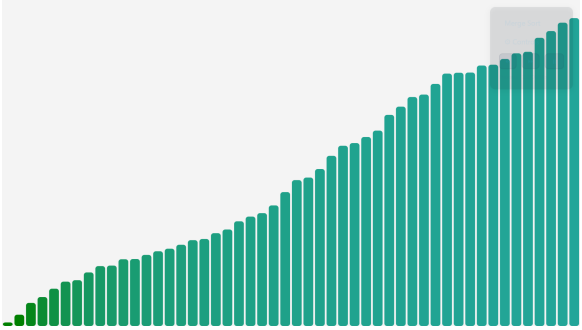
The following screenshots illustrate the step-by-step execution of **Merge Sort** on a randomly generated array. The visualizer highlights active comparisons and swaps in distinct colors, allowing the user to follow the algorithm's progress in real time.

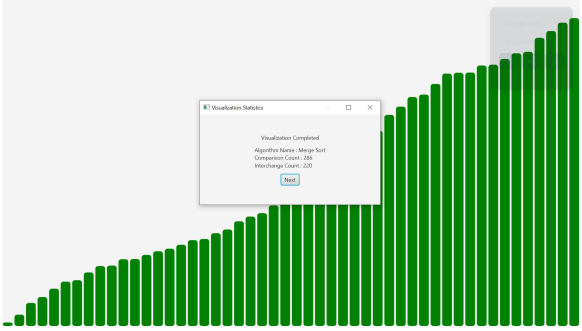






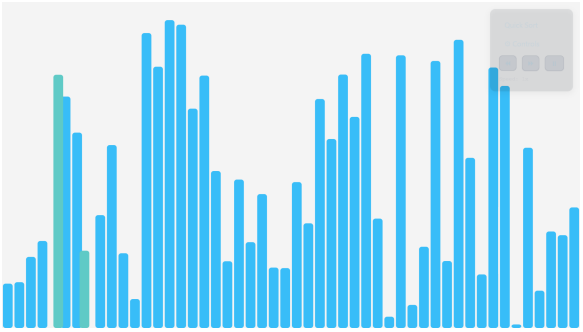


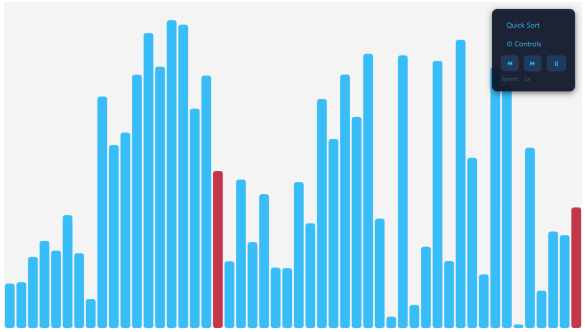


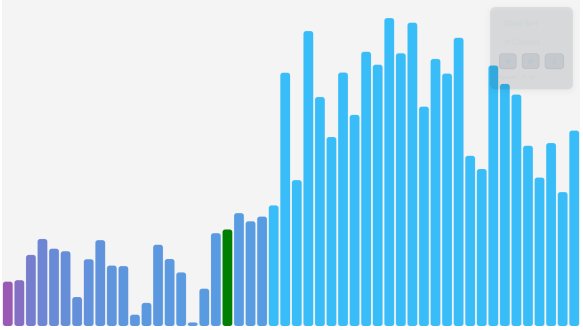


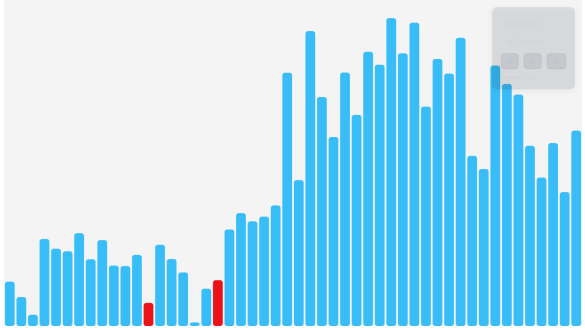
9.3 Quick Sort Visualization

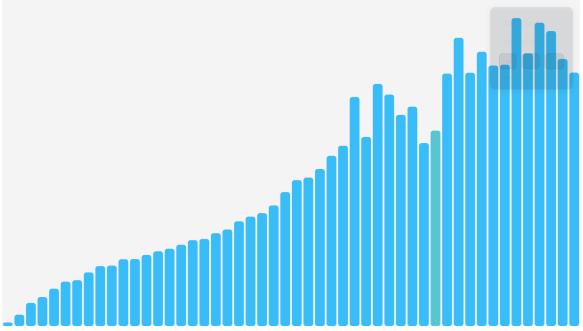
The following screenshots illustrate the execution of **Quick Sort** on a randomly generated array. The visualizer highlights the pivot element and active partition boundaries, allowing the user to observe how the algorithm divides and conquers the array.

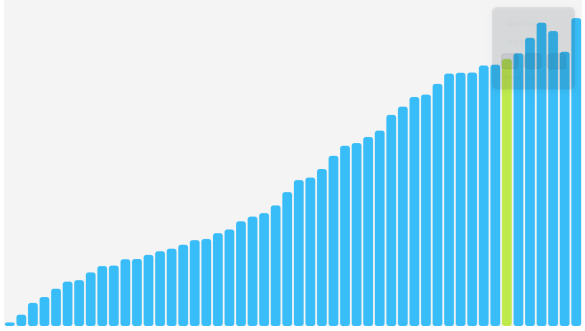


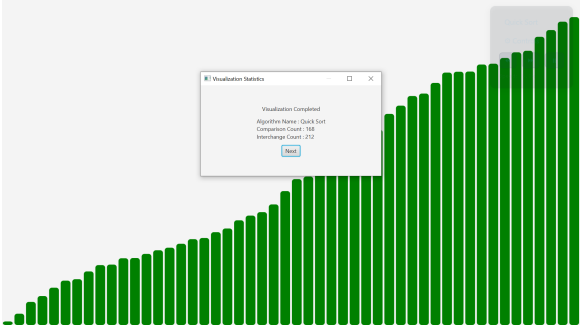












10 Conclusion

This assignment involved the design and implementation of a sorting algorithm visualizer following an **MVC-inspired architecture**. The system supports both visualization of sorting algorithms and systematic benchmarking across multiple input configurations.

10.1 Design and Implementation

The application was built around a clean separation of concerns. The **Strategy Pattern** allowed sorting algorithms to be implemented and extended independently. The **Observer Pattern** decoupled event generation from visualization and operation counting. The **event-driven approach** to visualization eliminated synchronization issues and enabled smooth, controllable animations with support for pause, resume, and reset functionality.

10.2 Experimental Findings

The empirical analysis conducted on 2160 observations across 6 algorithms, 4 input distributions, and 9 array sizes confirmed several theoretical expectations:

- Quadratic algorithms (**Bubble Sort**, **Selection Sort**, **Insertion Sort**) scale poorly for large inputs, with execution times growing rapidly as array size increases.
- $O(n \log n)$ algorithms (**Merge Sort**, **Heap Sort**, **Quick Sort**) demonstrated significantly better scaling behavior, with **Quick Sort** consistently outperforming all others for medium and large inputs.
- Input distribution has a measurable impact on quadratic algorithms, particularly **Insertion Sort**, which benefits greatly from nearly sorted inputs. In contrast, $O(n \log n)$ algorithms remained largely unaffected by input type.
- The log–log analysis empirically confirmed the theoretical complexity classes of all evaluated algorithms, with slopes closely matching their expected asymptotic behavior.

10.3 Lessons Learned

Several practical lessons were drawn from this assignment:

- Theoretical complexity alone does not fully determine practical performance. Constant factors, memory access patterns, and input characteristics all play a significant role.
- Reliable benchmarking requires repeated executions and careful aggregation to reduce the influence of system noise and JVM optimizations.
- Separating visualization logic from algorithmic logic through event-driven design significantly simplifies animation control and improves overall system maintainability.

Overall, this assignment provided both a practical implementation challenge and a deeper empirical understanding of sorting algorithm behavior across varying conditions.

End of Report